GROUP 2

Project Understanding

Freelance Invoicing and Payment Tracker

1. V D N Sathwik Yarlagadda
2. Siddharth Suddala
3. Shaik Guffran Ahmed

Purpose

A role-based workspace for freelancers and clients to manage project requests, time, review, invoicing, and demo payment records.

Prepared as a detailed project overview covering product understanding, workflow, architecture, and technical requirements.

1. Project overview

The application is a full-stack web application for a small freelance operation. It centralizes the work journey from an initial client request through project delivery, client review, invoicing, and payment tracking. It supports separate freelancer and client experiences while preserving ownership, privacy, and billing rules in the API.

The application is a demo ledger, not a payment gateway. It records cash or card payment entries for demonstration and accounting workflow purposes, but it does not process real card details or move money.

Primary user roles

ROLE	WHAT THE ROLE CAN DO	MAIN SCREENS / OUTCOMES
Freelancer	Manage clients, projects, time entries, invoices, payments, and the project board.	Dashboard, Clients, Projects, Time, Invoices, Payments, Project board.
Client	Post a project request, share a one-time code, review submitted work, see invoices, and record demo payments.	Client portal, Post Projects, Work review, Invoice detail, Payment history.

Functional scope

- Authentication and role-based authorization using access and refresh JWTs.
- Client, project, timesheet, invoice, payment, and dashboard management for the freelancer.

- Client project-post discovery and a one-time-code claim flow to safely convert a request into a working project.
- A delivery review workflow separated from the invoice and payment lifecycle.

2. End-to-end workflows

A. Project acquisition, delivery, and review

01	**Client posts work** The client publishes a title, scope, contact details, and positive hourly rate.
02	**Code is generated** The application creates a one-time project code. Only the owning client receives the raw code.
03	**Freelancer claims** The freelancer enters the code on the Project board. The app creates or reuses the client contact and creates a project at the offered rate.
04	**Time is logged** Time entries store the rate used when created, protecting prior charges from later rate changes.
05	**Work is submitted** The freelancer submits completion notes. The client either approves or requests a revision with feedback.
06	**Final invoice** After approval, the freelancer can create one final invoice from remaining unbilled hours. This closes the project workflow, while invoice status continues independently.

B. Invoice and payment lifecycle

01	**Build invoice** The freelancer creates an invoice from unbilled time entries. Server-side math calculates line values, subtotal, tax, and total.
02	**Send invoice** A draft becomes client-visible only after it is sent. Client portal queries omit drafts.
03	**Client pays** The client opens the invoice, enters a positive amount up to the outstanding balance, chooses Cash or Card, and can add a reference.
04	**Ledger updates** The API records the payment, updates amount paid, recalculates the balance, and displays the result to both roles.
05	**Status changes** The invoice becomes sent, partially paid, paid, or overdue according to payment amount and due date. A payment locks invoice edits to protect the audit trail.

3. Architecture and data model

The application uses a three-service container architecture. The browser runs a React single-page application, which calls a Flask REST API. The API owns validation, authorization, business calculations, and persistence in MongoDB.

React + Vite frontend	Flask API	MongoDB
React 18, React Router, Axios. Served on port 5173; development proxy forwards /api calls.	Flask blueprints for auth, freelancer, and client portal. Served on port 5000.	MongoDB 7 stores collections, counters, and blocked token IDs. Served on port 27017.

Core collections and relationships

AREA	KEY RECORDS	RELATIONSHIP / RESPONSIBILITY
Identity	users, token_blocklist	Users have unique email addresses, a role, password hash, and business name. Revoked JWT IDs are stored in the blocklist.
Work	clients, projects, time_entries, project_review_events	A freelancer owns client records. Clients have projects. Projects have time and review history.
Billing	invoices, invoice_line_items, payments	Invoices belong to clients and may be final for one project. Payments belong to invoices.
Project posts	project_posts, project_post_codes	Posts are client-owned discovery notices. Only a hashed one-time code is stored, and a successful claim links one project.

API design

All endpoints live below /api. Access checks are role-specific: freelancer endpoints protect owned records, while /portal endpoints use the authenticated client email to restrict visibility. List endpoints support page, per_page, sort, order, and selected filters.

4. Technical requirements

LAYER	REQUIRED TECHNOLOGY	IMPLEMENTATION NOTES
Frontend	React 18, Vite 6, React Router, Axios	Responsive client and freelancer views; Vite dev server on port 5173; Axios uses the API proxy.
Backend	Python 3.12+, Flask 3.1, Flask-Cors, Flask-JWT-Extended	REST API, JWT access/refresh flows, CORS origin control, role decorators, and centralized error responses.
Database	MongoDB 7, PyMongo	Collections use application-managed integer IDs, counters, indexes, pagination, and a persistent token blocklist.
Configuration	python-dotenv, environment variables	MONGO_URI, MONGO_DB, JWT_SECRET_KEY, and FRONTEND_ORIGIN are required configuration concerns.
Local runtime	Docker Desktop and Docker Compose, or MongoDB + uv + Node.js	Compose starts database, API, and web services. A named Mongo volume preserves data between restarts.

Security and business constraints

- Passwords are stored as salted PBKDF2-SHA256 hashes; responses do not expose password hashes.
- Access tokens expire after 20 minutes and refresh tokens after 14 days. Refresh rotation invalidates the old refresh token; logout blocklists tokens.
- CORS is restricted through FRONTEND_ORIGIN. Production must use a strong unique JWT secret, HTTPS, and a specific browser origin.
- Invoice totals are calculated server-side. Billed time cannot be edited, payments must be positive, and overpayments are rejected.
- Draft invoices are not exposed to clients. Invoice edits are locked after the first payment.
- Raw project codes are never stored; only hashes are persisted, and a code can be claimed once.

5. Delivery, testing, and acceptance criteria

Local setup

Container route: configure environment values, then run docker compose up --build. The browser application is available at http://localhost:5173 and the API health endpoint is http://localhost:5000/api/health. Local non-container route: sync the backend with uv, start Flask on port 5000, then run npm install and npm run dev in the frontend.

Verification approach

- Build the frontend with Vite to validate JSX, imports, and bundled browser assets.
- Run tests/smoke_routes.py against a running API to exercise public routes, roles, invoice calculations, locking, payment rules, refresh rotation, review workflow, code claim behavior, and access restrictions.
- Use the supplied Postman collection to manually validate endpoint payloads, statuses, and browser integration scenarios.
- Review the two roles in the UI: freelancer manages work and billing; client sees only their own sent invoices and linked project work.

Key acceptance criteria

AREA	ACCEPTANCE CRITERIA
Role separation	Unauthorized calls return 401; wrong roles return 403; client data is limited to the authenticated email and freelancer data to owned records.
Billing correctness	Invoice and payment amounts are calculated on the server, balances remain correct after partial payments, and paid/billed records are protected from invalid edits.
Review lifecycle	A client can approve or request revision only on submitted work; approval and invoice payment are independent states.
Project-code safety	Client receives raw code only at creation/regeneration; a claim creates or reuses the intended relationship and cannot be repeated.
Usability	The frontend shows clear invoice balances, role-appropriate actions, success/error states, and direct links between lists and detailed records.

Summary
The application is a carefully separated freelance-workflow application: work delivery is governed by review states, billing is governed by invoice states, and access is governed by role and ownership checks. Its technical design keeps financial calculations and authorization on the server while giving both roles a focused React experience.